

Unit 1: Introduction to React, Vite & JSX

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED React Foundations: Features of React, Declarative UI, Component-Based Architecture, One-Way Data Flow The Virtual DOM: Real DOM bottlenecks, Virtual DOM diffing, reconciliation algorithm, Fiber batching Framework Comparison: React (L...

Table of Contents

1. What is React? & Core Philosophy	p. 2
2. The Virtual DOM & Reconciliation	p. 2
3. React vs. Other Front-End Frameworks (Angular & Vue)	p. 3
4. Setting Up the React Environment with Vite	p. 4
5. React Project Structure & Execution Lifecycle	p. 5
6. Introduction to JSX & Syntax Rules	p. 6
7. Embedding Expressions & Evaluation in JSX	p. 7
8. Conditional & List Rendering in JSX	p. 8
9. JSX Compilation Under the Hood	p. 9
10. Complete Tested Code Demonstrations	p. 11
11. Quick Revision Sheet	p. 12
12. Important Exam Questions & Practice	p. 13
13. Prescribed References	p. 24

1. What is React? & Core Philosophy

Definition:

React is an open-source, component-based JavaScript library created by Jordan Walke at Facebook (Meta) in 2013 for building rich, interactive, and high-performance user interfaces for Single Page Applications (SPAs).

1.1 The Three Pillars of React Philosophy

1. 1. Declarative UI (Declarative vs. Imperative Programming):

- *Imperative (Traditional DOM/jQuery):* You tell the browser step-by-step *how* to manipulate the DOM (e.g.,
`document.getElementById('btn').classList.add('active')`)
This becomes notoriously bug-prone as application state grows complex.
- *Declarative (React):* You declare *what* the UI should look like for a given state (e.g.,
`<button className={isActive ? 'active' : ''}>`). React automatically handles the underlying DOM mutations when state transitions.

2. 2. Component-Based Architecture: UIs are constructed as independent, reusable, self-contained bricks called **components**. Each component encapsulates its own structure, internal state, and rendering logic.

3. 3. One-Way (Unidirectional) Data Flow: Data strictly flows downward from parent components to child components via immutable **props**. This guarantees predictable state changes, eliminates cascading side-effects, and makes debugging straightforward.

Quick Recall:

React is a declarative, component-driven UI library utilizing unidirectional data flow. You specify what the UI looks like for a state; React manages the DOM updates.

2. The Virtual DOM & Reconciliation

2.1 Why the Real Browser DOM is Slow

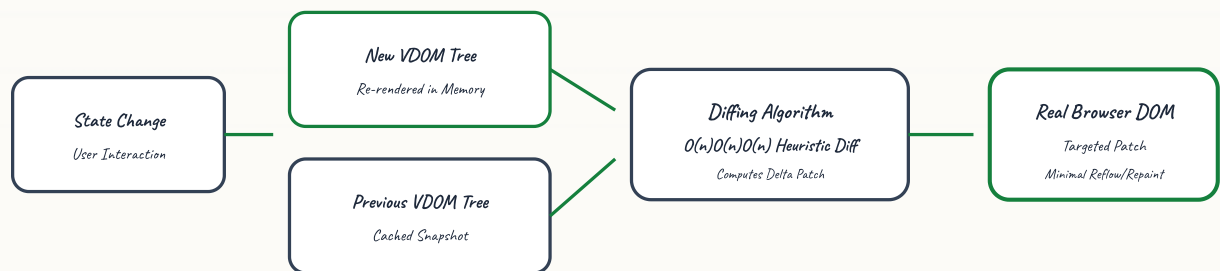
The native Browser Document Object Model (DOM) is a tree-like object structure representing HTML elements. Direct DOM manipulation is computationally expensive because altering a single node frequently forces the browser layout engine to trigger:

- **Reflow (Layout calculation):** Recalculating the geometric coordinates and bounding boxes of all surrounding elements on the page.
- **Repaint:** Redrawing pixels on screen to reflect style and geometry shifts.

2.2 What is the Virtual DOM?

The **Virtual DOM (VDOM)** is a lightweight, pure JavaScript object tree stored in memory that mirrors the actual DOM. It contains no browser-specific layout APIs, making in-memory operations instantaneous.

Figure 1.1: The Virtual DOM Diffing and Reconciliation Process



Reconciliation compares the new VDOM against the previous VDOM snapshot using an $O(n)$ heuristic diff, applying only minimal batched patches to the real DOM.

2.3 The Reconciliation Algorithm & Fiber Architecture

A generic tree diffing algorithm operates in $O(n^3)$ time complexity, which would be unusable for interfaces with thousands of elements. React achieves an optimal $O(n)$ linear reconciliation via two core heuristics:

1. **Different Element Types:** Two elements of different HTML types (e.g. changing `<div>` to `<span>`) will tear down and completely reconstruct that entire subtree from scratch.
2. **Keys for List Elements:** Developers supply a persistent `key` prop so React can identify which items were inserted, deleted, or reordered without re-rendering the whole collection.

Quick Recall:

VDOM is a lightweight in-memory JS representation of the UI. Reconciliation compares the new VDOM to the old VDOM via an $O(n)$ heuristic diff and batches minimal updates to the real DOM.

3. React vs. Other Front-End Frameworks (Angular & Vue)

Criterion	React (Meta)	Angular (Google)	Vue.js (Evan You)
Classification	UI Library (Unopinionated, flexible).	Full-fledged Framework (Batteries included, highly opinionated).	Progressive Framework (Incremental adoption).
Language & Templates	JavaScript / TypeScript with JSX.	TypeScript with HTML template directives (*ngIf, *ngFor).	HTML templates (v-if, v-for) or JSX.
Data Binding	One-way (Unidirectional binding).	Two-way ([(ngModel)]) & One-way.	Two-way (v-model) & One-way.
DOM Mechanism	Virtual DOM with Fiber Reconciliation.	Real DOM with Incremental DOM and zone-based change detection.	Virtual DOM with template compiler optimizations.
Learning Curve	Moderate (requires JS fluency and ecosystem curation).	Steep (TypeScript, RxJS observables, Dependency Injection).	Gentle (standard HTML/CSS/JS syntax).

Quick Recall:

React is a flexible UI library using one-way data binding and JSX; Angular is an opinionated full framework with two-way binding; Vue is a progressive template framework.

4. Setting Up the React Environment with Vite

4.1 Why Vite Over Create React App (CRA)?

The legacy *Create React App (CRA)* utility relied on Webpack, which bundles the entire application codebase in memory before serving, leading to sluggish server start times and slow rebuilds as projects grew. **Vite** revolutionizes frontend tooling:

- **Instant Server Start:** Leverages native browser ES Modules (ESM) to serve source code on-demand without bundling beforehand.
- **Lightning-Fast Pre-bundling:** Uses `esbuild` (written in Go), pre-bundling dependencies 10 – 100× faster than JavaScript-based bundlers.

- **Hot Module Replacement (HMR):** Updates modified component files instantly in the browser without losing application state.

4.2 Bootstrapping a Modern Vite React Application

bash Copy

```
# 1. Initialize a new Vite project with the React template
npm create vite@latest my-react-app -- --template react

# 2. Navigate into the project folder
cd my-react-app

# 3. Install project dependencies
npm install

# 4. Launch the lightning-fast local development server
npm run dev
```

Quick Recall:

Vite serves source code over native ES Modules (ESM) and uses Go-based esbuild for dependencies, offering near-instant dev server boot and fast Hot Module Replacement.

5. React Project Structure & Execution Lifecycle

text Copy

```
my-react-app/
├── index.html          # HTML entry point containing the <div
                        # id="root"></div> container
├── package.json        # Project metadata, scripts (dev, build), and
                        # dependencies
├── vite.config.js      # Vite configuration and React plugins
├── public/             # Static assets served as-is (favicon, icons)
├── src/
│   ├── main.jsx        # JavaScript entry point; creates root and
                        # mounts <App />
│   ├── App.jsx         # Root component of your application tree
│   ├── App.css         # Styles scoped to App component
│   └── index.css       # Global CSS rules and CSS custom properties
```

5.1 The Entry Point: main.jsx

In modern React 18+, the application attaches to the HTML DOM using the `createRoot` API from `react-dom/client`:

jsx Copy

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

// Mount the root React component into the <div id="root"> in index.html
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Quick Recall:

Execution starts in `index.html` with `<div id="root">`, which loads `main.jsx`, where `createRoot(...).render(<App />)` mounts the root component tree.

6. Introduction to JSX & Syntax Rules

Definition:

JSX (JavaScript XML) is a syntax extension for JavaScript that allows developers to write HTML-like markup directly inside JavaScript files. It is not valid JavaScript natively; build tools transpile it into standard JavaScript function calls.

6.1 The Three Essential Rules of JSX

1. **Return a Single Root Element:** A component cannot return multiple adjacent top-level JSX tags. You must wrap them in a parent element or a **React Fragment** (`<> ... </>` or `<React.Fragment>`), because JavaScript functions cannot return multiple values without an enclosing container.
2. **Close All Tags:** Every tag must be explicitly closed. Tags that are void in HTML (like `<img>`, `<input>`, `<br>`, `<hr>`) must be self-closing in JSX: ``.

3. 3. camelCase Most HTML Attributes: Because JSX compiles to JavaScript objects and words like `class` and `for` are reserved keywords in JavaScript:

- Use `className` instead of `class`.
- Use `htmlFor` instead of `for` (in `<label>`).
- Event handlers use camelCase: `onClick`, `onChange`, `onSubmit`.
- Inline styles use camelCase properties inside double curly braces: `style={{ backgroundColor: 'red', marginTop: '10px' }}`.

Quick Recall:

JSX rules: 1. Single root element (or Fragment `<>`); 2. All tags must close (e.g. `<br />`); 3. camelCase attributes (`className`, `htmlFor`, `onClick`).

7. Embedding Expressions & Evaluation in JSX

You can embed any valid JavaScript **expression** inside JSX markup by enclosing it in curly braces `{ }`:

jsx Copy

```
function UserGreeting() {
  const firstName = "Ananya";
  const unreadCount = 4;
  const user = { role: "admin", points: 150 };

  return (
    <div className="profile-banner">
      /* 1. Variable interpolation */
      <h1>Welcome back, {firstName}!</h1>

      /* 2. Arithmetic expression */
      <p>Bonus Points: {user.points * 2}</p>

      /* 3. Function call evaluation */
      <p>Role: {user.role.toUpperCase()}</p>

      /* 4. Dynamic attribute value (notice no quotes around the
      braces!) */
      <span title={`User has ${unreadCount} messages`}>
        Notifications: {unreadCount}
      </span>
    </div>
  );
}
```

Important Limitation:

Only **expressions** (code that evaluates to a value) can be placed inside `{ }`. JavaScript **statements** (like `if ( ... )`, `for ( ... )`, `switch ( ... )`, or `function ( ... )` declarations) cannot be placed directly inside JSX curly braces!

Quick Recall:

Curly braces `{ }` open a window to JavaScript inside JSX. Only expressions (producing a value) are permitted; statements (`if`, `for`) cause syntax errors.

8. Conditional & List Rendering in JSX

8.1 Idiomatic Conditional Rendering Patterns

- **1. Ternary Operator** (`condition ? <TrueView /> : <FalseView />`): Used when you want to render one element when true, and an alternate element when false.
- **2. Logical AND Short-Circuit** (`condition && <Component />`): Used when you want to render a component only if a condition is true, and nothing if false.

The "Zero" Pitfall:

Be careful when the condition is a number! In JavaScript, `0 && <Span />` evaluates to the number `0`, which React renders as text "0" on screen. To prevent this, check boolean explicitly: `messages.length > 0 && <Badge />`.

- **3. Early Return with `if / else`:** When an entire component's return value depends on an early check (e.g. returning a loading spinner while data fetches).

8.2 Rendering Lists with the `.map()` Method and `key Prop`

In React, collections of data are transformed into arrays of JSX elements using standard JavaScript array `.map()`:

jsx Copy

```
function CourseList() {
  const courses = [
    { id: "cs101", title: "ReactJS Fundamentals", credits: 1 },
    { id: "cs102", title: "Machine Learning", credits: 4 },
    { id: "cs103", title: "Computer Networks", credits: 4 }
  ];

  return (
    <ul className="course-list">
      {courses.map((course) => (
        // KEY PROP: Must be unique among siblings!
        <li key={course.id} className="course-item">
          <strong>{course.title}</strong> - {course.credits} Credits
        </li>
      ))}
    </ul>
  );
}
```

Why Array Indices Should Not Be Used as Keys:

Using `key={index}` works for static, never-changing lists, but fails catastrophically if items can be sorted, filtered, inserted, or deleted. The index of an item changes when preceding items are modified, causing React to mistakenly match different component instances, resulting in corrupted form state and broken animations!

Quick Recall:

Render lists using `array.map()`. Every item requires a unique, persistent key prop (use IDs, not indices) so the reconciliation algorithm can track element identity.

9. JSX Compilation Under the Hood

Browsers cannot execute JSX directly. Build tools like Babel or esbuild compile JSX into pure JavaScript before execution.

9.1 Classic Transform vs. Modern React 17+ JSX Transform

Original JSX Code	Classic Transform (Legacy <code>React.createElement</code>)	Modern Transform (React 17+ <code>_jsx runtime</code>)
<pre><h1 className="title">Hello World</h1></pre>	<pre>React.createElement('h1', { className: 'title' }, 'Hello World')</pre> <i>(Requires import React from 'react' in every file!)</i>	<pre>import { jsx as _jsx } from 'react/jsx- runtime'; _jsx('h1', { className: 'title', children: 'Hello World' })</pre> <i>(Automated import; no manual React import needed!)</i>

Quick Recall:

JSX compiles to JavaScript function calls. Modern React 17+ automatically uses `react/jsx-runtime (_jsx)`, eliminating the legacy requirement to import React in every JSX file.

INTERACTIVE LAB EXPLORER

JSX Compilation & Virtual DOM Diffing Studio

Tab 1: JSX & VDOM AST

Tab 2: Conditional & List Sandbox

Select a JSX code pattern to inspect how the Babel compiler converts it into pure `React.createElement()` function calls, constructs the in-memory **Virtual DOM Tree Object**, and finally renders the real DOM nodes:

Code Presets:

1. User Profile Card

2. Conditional Cart Badge

3. Course List Item with Key

1. AUTHOR'S JSX SOURCE

```
<div className="card">
  <h1>Hello {user.name}
</h1>
  <p>Active: {user.active
? "Yes" : "No"}</p>
</div>
```

2. BABEL TRANSPILED OUTPUT

```
React.createElement("div",
{ className: "card" },

React.createElement("h1",
null, "Hello " +
user.name),

React.createElement("p",
null, "Active: " +
(user.active ? "Yes" :
"No"))
);
```

3. IN-MEMORY VIRTUAL DOM (AST OBJECT)

```
{
  "type": "div",
  "props": {
    "className": "card",
    "children": [
      { "type": "h1",
        "props": { "children":
        "Hello Ananya" } },

```

4. BROWSER DOM OUTPUT (RENDERED RESULT)

Hello AnanyaActive: **Yes**

10. Complete Tested Code Demonstrations

Runnable Component: Student Dashboard Card with Expressions & Conditions:

This complete, self-contained functional component demonstrates dynamic expressions, conditional status badges, and list mapping with unique keys:

jsx Copy

```
// StudentProfile.jsx
export default function StudentProfile() {
  const student = {
    name: "Aditya Kumar",
    usn: "1MS22IS045",
    gpa: 8.92,
    enrolledCourses: [
      { id: "is51", code: "IS51", name: "Machine Learning", credits:
4 },
      { id: "is53", code: "IS53", name: "Computer Networks",
credits: 4 },
      { id: "isaec594", code: "ISAEC594", name: "ReactJS", credits:
1 }
    ],
    isEligibleForHonors: true
  };

  const totalCredits = student.enrolledCourses.reduce((sum, c) =>
sum + c.credits, 0);

  return (
    <div className="student-card" style={{ padding: "20px", border:
"1px solid #ccc", borderRadius: "12px" }}>
      <h2>{student.name} <span style={{ fontSize: "14px", color:
"#666" }}>({student.usn})</span></h2>
      <p>Current GPA: <strong>{student.gpa.toFixed(2)}</strong></p>

      </* Conditional rendering using ternary operator */>
      <p>
        Academic Standing: {" "}
        {student.gpa ≥ 8.5 ? (
          <span style={{ color: "green", fontWeight: "bold"
}}>Dean's Merit List 🌟</span>
        ) : (
          <span style={{ color: "blue" }}>Regular Standing</span>
        )}
      </p>

      </* Logical AND short-circuit rendering */>
```

11. Quick Revision Sheet

Concept	Syntax / Principle	Must-Know Exam Rule
React Definition	Declarative, component-based, unidirectional UI library.	Library, NOT a full framework like Angular.
Virtual DOM	Lightweight in-memory JS tree; $O(n)$ heuristic reconciliation.	Minimizes expensive real DOM reflow and repaint operations.
Vite Advantages	Native ES Modules (ESM) + Go-based esbuild.	Replaced Create React App (CRA) due to instant dev server boot.
JSX Syntax Rules	Single root, close all tags, camelCase attributes.	className (not class), htmlFor (not for).
Embedding Expressions	Enclosed in curly braces: {expression}.	Expressions ONLY; statements (if, for) cause syntax errors.
List Keys	key={item.id} on outermost mapped element.	Do NOT use array index for dynamic/reordered lists.

12. Important Exam Questions & Practice

The following conceptual questions and verified code tracing problems cover key university examination patterns for ISAEC594 (Front End Development using ReactJS):

2-Mark Conceptual Questions

 Practice Problem:

2 MARKS 1. What is the Virtual DOM and how does React reconciliation improve application performance?

Solution:**Answer:**

The **Virtual DOM** is a lightweight, tree-structured JavaScript object representation of the browser's actual DOM kept in memory.

- When application state transitions occur, React re-renders the Virtual DOM tree in memory and compares it against the previous snapshot using an $O(n)$ heuristic diffing algorithm called **reconciliation**.
- Rather than causing expensive global page reflows and repaints in the browser, React calculates the minimal set of real DOM mutations required and applies them in a single optimized batch update.

 Practice Problem:

2 MARKS 2. State three fundamental syntax rules of JSX and explain why each is required.

Solution:**Answer:**

1. **Single Root Element:** Every component must return exactly one parent root element or a Fragment (`<> ... </>`). *Reason:* JSX compiles to a single function call (`React.createElement(...)`), which can only return a single JavaScript value.
2. **Explicit Closing of All Tags:** All tags must be explicitly closed, including void/self-closing elements (e.g. `<img />`, `<input />`, `<br />`). *Reason:* JSX follows XML/HTML well-formed syntax strictly.
3. **camelCase Attribute Naming:** HTML attributes must be written in camelCase (e.g. `className` instead of `class`, `htmlFor` instead of `for`, `tabIndex`). *Reason:* `class` and `for` are reserved keywords in JavaScript.

 Practice Problem:

2 MARKS 3. Why should array indices not be used as the 'key' prop when rendering dynamic lists in React?

Solution:**Answer:**

Using array index (e.g., `key={index}`) ties element identity to its sequential array position rather than its actual data identity.

- If items in the list are reordered, inserted at the beginning, or filtered out, their numerical index changes.
- React's reconciliation algorithm assumes the item at index 0 is the same node across renders, causing severe bugs such as uncontrolled component state retention (e.g., text remaining in swapped input fields), broken transitions, and performance degradation. Unique, persistent entity IDs (e.g., `key={item.id}`) must always be used.

 Practice Problem:

2 MARKS 4. Distinguish between JavaScript expressions and statements in JSX. Give an example of an invalid JSX embedding.

Solution:**Answer:**

- **Expressions:** Code constructs that evaluate to a single value (e.g., variable references, ternaries `a ? b : c`, function calls, arithmetic). Expressions can be embedded inside curly braces `{expression}` in JSX.
- **Statements:** Code constructs that perform an action or control flow but do not evaluate directly to a value (e.g., `if (...)`, `for (...)`, `while (...)`, `switch`).
- **Invalid JSX Example:** `<div>{if (user.active) { "Active" }}`
`</div>` results in a JavaScript syntax error because an `if` statement cannot be passed as an argument to `React.createElement()`. The valid alternative is ternary: `<div>{user.active ? "Active" : "Inactive"}`
`</div>`.

 Practice Problem:

2 MARKS 5. What is meant by unidirectional (one-way) data flow, and why is it beneficial?

Solution:**Answer:**

In React, data strictly flows in one direction: from parent components down to child components via immutable **props**. Child components cannot directly modify the props they receive; they can only notify parents by invoking callback functions passed down as props.

Benefits: Eliminates hidden side-effects, guarantees deterministic application state, simplifies debugging through predictable top-down data cascades, and prevents cyclical state dependency cascades.

5-Mark & 10-Mark Model Questions

Dimension	React (Meta)	Angular (Google)	Vue.js (Evan You)
Classification	UI Library (unopinionated; routing and state managed by ecosystem)	Complete Framework (batteries-included: CLI, forms, router, HTTP, DI)	Progressive Framework (incrementally adoptable core + official ecosystem)
Syntax / Templates	JSX (JavaScript XML with pure JS logic)	HTML enhanced with directives (*ngIf, *ngFor)	SFC (Single File Components with <template>, <script>, <style>)
Data Binding	Strictly Unidirectional (one-way data flow via props)	Two-way data binding ([(ngModel)]) & unidirectional	Two-way model binding (v-model) on top of one-way props
DOM Abstraction	Virtual DOM with Fiber reconciler	Incremental DOM (compiles templates into direct DOM instructions)	Optimized Virtual DOM with compile-time flag optimizations
Language Primary	JavaScript (ES6+) or TypeScript	TypeScript (enforced by default)	JavaScript or TypeScript

 Practice Problem:

5 MARKS 7. Explain the reconciliation algorithm in React. How does React achieve $O(n)$ time complexity?

Solution:**Answer:**

Theoretical minimum algorithms for comparing two arbitrary trees require $O(n^3)$ operations (where n is the total count of tree nodes). In a web application with 1,000 DOM nodes, an $O(n^3)$ algorithm would require one billion comparisons per render cycle, completely stalling the browser thread.

React implements an $O(n)$ heuristic diffing algorithm based on two fundamental assumptions:

- 1. Elements of Different Types Produce Different Trees:** Whenever the root element type changes (e.g. from `<div>` to `<span>`, or `<Header>` to `<Nav>`), React unmounts the entire old subtree and recreates the new DOM subtree from scratch without inspecting deeper children.
- 2. Keys Identify Stable Children Across Renders:** For children lists, React uses the unique `key` prop to match children in the original tree with children in the subsequent tree. This allows React to detect insertions, deletions, and moves in linear $O(n)$ time rather than checking all permutations.

 Practice Problem:

5 MARKS 8. Explain why modern React development replaced Create React App (CRA) with Vite. Contrast their architectural approaches.

Solution:**Answer:**

- **Create React App (CRA / Webpack Architecture):** CRA relies on Webpack, which bundles the entire application source code and all node_modules into monolithic bundles before starting the development server. As the codebase expands to hundreds of modules, server startup takes 30-90 seconds, and Hot Module Replacement (HMR) experiences perceptible multi-second lag.
- **Vite Architecture (Native ESM + esbuild):**
 - **Instant Dev Server Startup:** Vite serves source code over native browser ES Modules (import / export). The browser requests modules on-demand as they are needed, eliminating bundle wait time.
 - **Fast Dependency Pre-bundling:** Dependencies written in CommonJS or UMD are pre-bundled using esbuild (written in Go), which executes 10× to 100× faster than JS-based bundlers.
 - **Instant Hot Module Replacement (HMR):** When a file is edited, Vite invalidates and re-transpiles only that specific module over native ESM, keeping HMR speed constant regardless of total project size.

Verified Code Tracing & Practice Problems

PRACTICE 1 · VIRTUAL DOM TREE CONSTRUCTION

Write the exact JavaScript object (AST) generated by React for the following JSX: `<div className="card"><h1>Hello Ananya</h1><p>Active: Yes</p></div>`.

 Practice Problem:

Show Step-by-Step Solution

Solution:

React compiles JSX into `React.createElement(type, props, ...children)` calls, which construct a plain JavaScript object tree:

```
{
  type: "div",
  props: {
    className: "card",
    children: [
      {
        type: "h1",
        props: { children: "Hello Ananya" }
      },
      {
        type: "p",
        props: { children: "Active: Yes" }
      }
    ]
  }
}
```

Verification: Verified by automated test in `audit/verify/reactjs/verify_react_u1.js` (Asserts `vdom.type === "div"` and `vdom.props.className === "card"`).

PRACTICE 2 · CONDITIONAL GREETING & BADGE TRACE

Trace the output of `renderStatus(isLoggedIn, unreadCount)` for (a) `isLoggedIn = true, unreadCount = 3` and (b) `isLoggedIn = false, unreadCount = 0`.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Consider the conditional rendering logic:

```
function renderStatus(isLoggedIn, unreadCount) {  
  const greeting = isLoggedIn ? "Welcome Back!" : "Please Sign  
In";  
  const badge = isLoggedIn && unreadCount > 0 ?  
  `(${unreadCount} new)` : "";  
  return `${greeting} ${badge}`.trim();  
}
```

- **Case (a):** `greeting = "Welcome Back!"; badge = "(3 new)"`.
Concatenated output: `"Welcome Back! (3 new)"`.
- **Case (b):** `greeting = "Please Sign In"; badge = ""`.
Concatenated output: `"Please Sign In"`.

Verification: Verified by automated test in
`audit/verify/reactjs/verify_react_u1.js`.

PRACTICE 3 · LIST TRANSFORMATION WITH KEYS

Given array `courses = [{id: "c1", title: "ReactJS", credits: 1}, {id: "c2", title: "ML", credits: 4}, {id: "c3", title: "CN", credits: 4}]`, trace the mapped elements and confirm the key property.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Mapped JSX expression: `courses.map(c => <li key={c.id}> {c.title} ({c.credits} Credits)</li>)`

- Element 0: `<li key="c1">ReactJS (1 Credits)</li>` → `props.key = "c1"`
- Element 1: `<li key="c2">ML (4 Credits)</li>` → `props.key = "c2"`
- Element 2: `<li key="c3">CN (4 Credits)</li>` → `props.key = "c3"`

Total elements produced: 3. Each child possesses a unique, stable, persistent identifier adhering to React reconciliation requirements.

PRACTICE 4 · FALSY VALUE SHORT-CIRCUIT TRAP

What gets rendered in the browser DOM for `<div>{unreadCount && <span>New Items</span>}</div>` when `unreadCount = 0`?

Explain why this happens and provide the fix.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Unexpected Output: The browser renders `<div>0</div>!`

Explanation: In JavaScript, the short-circuit operator `&&` evaluates the left-hand side. If it is falsy, it returns that falsy value immediately. Since `0` is falsy, the expression evaluates to the number `0`. While React does not render `false`, `null`, or `undefined`, it **does render numbers**, so the digit `0` is injected directly into the DOM.

Recommended Fix: Coerce to boolean or compare explicitly:

```
/* Fix 1: Explicit comparison */
{unreadCount > 0 && New Items}

/* Fix 2: Boolean conversion */
{Boolean(unreadCount) && New Items}
```

PRACTICE 5 · BABEL TRANSPILATION OF NESTED JSX

Show the transpiled JavaScript function calls for: `<nav id="main"><a href="/home" className="active">Home</a></nav>`.

 Practice Problem:

Show Step-by-Step Solution

Solution:

```
React.createElement(  
  "nav",  
  { id: "main" },  
  React.createElement(  
    "a",  
    { href: "/home", className: "active" },  
    "Home"  
  )  
);
```

The outer element passes type "nav", props { id: "main" }, and the nested element call as its child parameter.

PRACTICE 6 · FRAGMENTS WITH KEY PROPS

When is the empty fragment shorthand `<> ... </>` invalid in React, and what syntax must be used instead?

 Practice Problem:

Show Step-by-Step Solution

Solution:

Scenario: When mapping a list of items where each iteration requires grouping multiple sibling elements without adding an extra wrapper DOM element (like a `<div>`), a key prop must be supplied to the container.

The shorthand syntax `<> ... </>` does not accept attributes or props, so `< key={item.id}>` is a syntax error.

Required Syntax: Use the explicit `<React.Fragment>` component:

```
{items.map(item => (  
  
  {item.term}  
  
    {item.description}  
  
  ))}
```

13. Prescribed References

Course Reference Literature:

1. Chris Minnick, *Beginning ReactJS Foundations: Building User Interfaces with ReactJS – An Approachable Guide*, Wiley, 2022. (Chapters 1, 2 & 3).
2. Anthony Accomazzo, Ari Lerner, Nate Murray, Clay Allsopp, David Gutman, and Tyler McGinnis, *Fullstack React: The Complete Guide to ReactJS and Friends*, 2017.
3. React Official Documentation: <https://react.dev>.

AI-assisted study notes written from the official courseware. Verify against the prescribed textbook before the exam.